Evolving genpd's ->sync_state() support

Ulf Hansson, Linaro
Saravana Kannan



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Why ->sync_state()?

- Bootloader power-on things that must stay on until drivers have probed.
- A timeout doesn't fit when no driver or modular kernels.
- The ->sync_state() mechanism with **fw_devlink** provides a unified solution.

Deployment to generic PM domain (genpd)

In v6.17 we landed ->sync_state() support to genpd.

- Quite some regressions reported/fixed in fw_devlink, genpd and drivers.
- Some unused genpds stays powered-on.
- Consumer tracking of multi-provider genpds needs improvements.
- Potential to act as a reference for other subsystems.

1/2: Some unused genpds stays powered-on

No timeout since FW_DEVLINK_SYNC_STATE_STRICT is default.

- Enforce ->sync_state() for a device/driver via sysfs.
- Enable timeout via kernel cmdline or Kconfig.
- Call **of_genpd_sync_state()** from the genpd provider.
- Set **GENPD_FLAG_NO_STAY_ON** to opt out from new behaviour.

2/2: Some unused genpds stays powered-on

How can we address the problem?

- Can we make **FW_DEVLINK_SYNC_STATE_TIMEOUT** default?

- Use the “probe defer timeout” to make a per genpd decision?

1/2: Unused genpds stays on for a multi-genpd provider

power-domain-cells = <1> to specify the domain-id, like for Arm SCMI.

- A missing consumer driver causes unused genpds to stay powered-on.
- Without a “sync_state” timeout the problem increases.
- Would this be a potential problem for other subsystems too?

2/2: Unused genpds stays on for a multi-genpd provider

How can we address the problem?

- A fine-grained solution to “sync_state” on a per domain-id/genpd basis?
- Multiple devices sharing the same fwnode doesn’t play with fw_devlink.
- Of-parse at genpd registration for consumers on per domain-id basis, to allow tracking them during attach? Does fw_devlink provide some of the data?